

LAB3-24 · 문자 LCD 제어

전자전기컴퓨터설계실험II · FPGA 응용 실습

HD44780 호환 LCD를 8비트 write-only 방식으로 초기화하고 두 줄 문자열을 표시한다.

Vivado 2026.1 · FPGA Part xc7s75fgga484-1

작성일 2026. 09. 21. · 이해리

목차

동작과 검증 목표

v2.0.2 프로젝트 시작

RTL·TB·XDC 직접 작성

VS Code·Icarus 시뮬레이션

Vivado XSim·비트스트림

보드 실험과 레포트

이번 회로의 목표

HD44780 호환 LCD를 8비트 write-only 방식으로 초기화하고 두 줄 문자열을 표시한다.

핵심 개념: 초기화 명령, Enable setup/high/hold, DDRAM 주소, ASCII, 명령별 대기

기능 TB: `tb_character_lcd`

예상 PASS: `LAB3_LCD_PASS` bytes=40

입출력과 보드 연결

clk_50mhz B6 · rst_p K4 · lcd_e A6 · lcd_rs G6 · lcd_rw D6 ·
lcd_data[7:0] A4/B2/C3/D4/A2/C5/C1/D1

모든 순차 로직은 MAIN CLOCK F의 50 MHz만 사용한다. 느린 동작은 내부 clock-enable로 만든다.

XDC의 포트 이름은 top module 선언과 글자·대소문자까지 같아야 한다.

TB가 자동으로 확인하는 범위

Enable 하강 에지에서 초기화 명령과 두 줄 데이터, 총 40바이트의 순서와 RS를 검사한다.

시뮬레이션 파라미터는 실제 동작의 순서와 경계 조건을 유지하면서 시간을 줄인다.

PASS는 디지털 기능 검사 결과다. 핀 배선, 전기 규격, 실제 부품의 물리 동작은 보드 단계에서 확인한다.

v2.0.2 템플릿을 새 이름으로 clone

PowerShell에서 한 줄씩 실행한다.

```
1 git clone --branch v2.0.2 https://github.com/Glaysia/fpga-lab-template.git lab3_24_character_lcd
2 cd lab3_24_character_lcd
3 git switch -c main
4 code FPGA.code-workspace
```

태그 clone 직후 detached HEAD 안내는 정상이다. git switch -c main으로 자신의 이력을 시작한다.

빈 템플릿에서 만들 파일

경로	파일
src/	lab3_character_lcd.v
sim/	tb_character_lcd.sv
constraints/	lab3_character_lcd.xdc
프로젝트 루트	simulation.json

기존 빈 파일은 이름을 바꾸고, 필요한 추가 RTL은 src에 새로 만든다.

simulation.json 확인

sources에는 모든 RTL 파일을 의존 순서에 맞춰 넣는다.

testbench는 sim/tb_character_lcd.sv, simulation_top은 tb_character_lcd다.

설계 top lab3_character_lcd은 Vivado 합성에서 선택한다. TB module을 합성 top으로 선택하지 않는다.

코드를 읽고 입력하는 기준

다음 코드 화면은 줄 번호와 파일명이 원본 프로젝트와 연결되어 있다.

RTL은 합성 가능한 회로, TB는 입력 자극과 기대값 비교, XDC는 핀과 20 ns 클록 제약을 담당한다.

코드를 붙여 넣는 것으로 끝내지 말고 counter 폭, 파라미터 계산, 리셋 우선순위, 출력 시점을 설명할 수 있어야 한다.

simulation.json · 1-5행

```
1 {  
2   "sources": ["src/lab3_character_lcd.v"],  
3   "testbench": "sim/tb_character_lcd.sv",  
4   "simulation_top": "tb_character_lcd"  
5 }
```

lab3_character_lcd.v · 1-18행

```
1 `timescale 1ns/1ps
2 module lab3_character_lcd #(
3     parameter integer TICK_CYCLES = 500,
4     parameter integer POWER_TICKS = 2_000,
5     parameter integer NORMAL_WAIT_TICKS = 4,
6     parameter integer CLEAR_WAIT_TICKS = 200
7 ) (
8     input wire clk_50mhz,
9     input wire rst_p,
10    output reg lcd_e,
11    output reg lcd_rs,
12    output wire lcd_rw,
13    output reg [7:0] lcd_data
14 );
15    localparam integer TICK_WIDTH = (TICK_CYCLES < 2) ? 1 : $clog2(TICK_CYCLES);
16    reg [TICK_WIDTH-1:0] tick_count;
17    reg tick;
18    reg [1:0] phase;
```

lab3_character_lcd.v · 19–36행

```
19  reg [5:0] index;
20  integer wait_count;
21  integer power_count;
22  reg byte_rs;
23  reg [7:0] byte_data;
24  assign lcd_rw = 1'b0;
25
26  always @(posedge clk_50mhz or posedge rst_p) begin
27      if (rst_p) begin
28          tick_count <= 0;
29          tick <= 0;
30      end else if (tick_count == TICK_CYCLES - 1) begin
31          tick_count <= 0;
32          tick <= 1;
33      end else begin
34          tick_count <= tick_count + 1'b1;
35          tick <= 0;
36      end
```

lab3_character_lcd.v · 37-54행

```
37 end
38
39 always @* begin
40     byte_rs = 1'b0;
41     byte_data = 8'h00;
42     case (index)
43         0, 1, 2: byte_data = 8'h38;
44         3: byte_data = 8'h0c;
45         4: byte_data = 8'h06;
46         5: byte_data = 8'h01;
47         6: byte_data = 8'h80;
48         7: begin byte_rs=1; byte_data="F"; end
49         8: begin byte_rs=1; byte_data="P"; end
50         9: begin byte_rs=1; byte_data="G"; end
51         10: begin byte_rs=1; byte_data="A"; end
52         11: begin byte_rs=1; byte_data=" "; end
53         12: begin byte_rs=1; byte_data="L"; end
54         13: begin byte_rs=1; byte_data="A"; end
```

lab3_character_lcd.v · 55-72행

```
55      14: begin byte_rs=1; byte_data="B"; end
56      15: begin byte_rs=1; byte_data="3"; end
57      16,17,18,19,20,21,22: begin byte_rs=1; byte_data=" "; end
58      23: byte_data = 8'hc0;
59      24: begin byte_rs=1; byte_data="L"; end
60      25: begin byte_rs=1; byte_data="C"; end
61      26: begin byte_rs=1; byte_data="D"; end
62      27: begin byte_rs=1; byte_data=" "; end
63      28: begin byte_rs=1; byte_data="C"; end
64      29: begin byte_rs=1; byte_data="0"; end
65      30: begin byte_rs=1; byte_data="N"; end
66      31: begin byte_rs=1; byte_data="T"; end
67      32: begin byte_rs=1; byte_data="R"; end
68      33: begin byte_rs=1; byte_data="0"; end
69      34: begin byte_rs=1; byte_data="L"; end
70      35: begin byte_rs=1; byte_data="L"; end
71      36: begin byte_rs=1; byte_data="E"; end
72      37: begin byte_rs=1; byte_data="R"; end
```

lab3_character_lcd.v · 73-90행

```
73         38,39: begin byte_rs=1; byte_data=" "; end
74         default: byte_data = 8'h00;
75     endcase
76 end
77
78 always @(posedge clk_50mhz or posedge rst_p) begin
79     if (rst_p) begin
80         lcd_e <= 0; lcd_rs <= 0; lcd_data <= 0;
81         phase <= 0; index <= 0; wait_count <= 0; power_count <= 0;
82     end else if (tick) begin
83         if (power_count < POWER_TICKS) begin
84             power_count <= power_count + 1;
85             lcd_e <= 0;
86         end else begin
87             case (phase)
88                 0: begin
89                     lcd_e <= 0; lcd_rs <= byte_rs; lcd_data <= byte_data;
90                     phase <= 1;
```

lab3_character_lcd.v · 91-108행

```
91         end
92         1: begin lcd_e <= 1; phase <= 2; end
93         2: begin
94             lcd_e <= 0;
95             wait_count <= (index == 5) ? CLEAR_WAIT_TICKS : NORMAL_WAIT_TICKS;
96             phase <= 3;
97         end
98         default: begin
99             if (wait_count > 0)
100                 wait_count <= wait_count - 1;
101             else begin
102                 index <= (index == 39) ? 6 : index + 1'b1;
103                 phase <= 0;
104             end
105         end
106     endcase
107 end
108 end
```

lab3_character_lcd.v · 109–110행

```
109     end  
110 endmodule
```

tb_character_lcd.sv · 1-18행

```
1 `timescale 1ns/1ps
2 module tb_character_lcd;
3     reg clk_50mhz=0,rst_p=1;
4     wire lcd_e,lcd_rs,lcd_rw;
5     wire [7:0] lcd_data;
6     reg [7:0] expected_data[0:39];
7     reg expected_rs[0:39];
8     integer count=0,i;
9     always #10 clk_50mhz=~clk_50mhz;
10    lab3_character_lcd #(.TICK_CYCLES(2),.POWER_TICKS(3),.NORMAL_WAIT_TICKS(1),.CLEAR_WAIT_TICKS(2)
11        ) dut
12        (.clk_50mhz(clk_50mhz),.rst_p(rst_p),.lcd_e(lcd_e),.lcd_rs(lcd_rs),.lcd_rw(lcd_rw),.lcd_data(
13            lcd_data));
14
15    initial begin
16        expected_data[0]=8'h38;expected_data[1]=8'h38;expected_data[2]=8'h38;
17        expected_data[3]=8'h0c;expected_data[4]=8'h06;expected_data[5]=8'h01;expected_data[6]=8'h80;
18        expected_data[7]="F";expected_data[8]="P";expected_data[9]="G";expected_data[10]="A";
19        expected_data[11]=" ";expected_data[12]="L";expected_data[13]="A";expected_data[14]="B";
20        expected_data[15]="3";
21        for(i=16;i<=22;i=i+1)expected_data[i]=" ";
```

tb_character_lcd.v · 19-36행

```
19     expected_data[23]=8'h0;
20     expected_data[24]="L";expected_data[25]="C";expected_data[26]="D";expected_data[27]=" ";
21     expected_data[28]="C";expected_data[29]="0";expected_data[30]="N";expected_data[31]="T";
22     expected_data[32]="R";expected_data[33]="0";expected_data[34]="L";expected_data[35]="L";
23     expected_data[36]="E";expected_data[37]="R";expected_data[38]=" ";expected_data[39]=" ";
24     for(i=0;i<40;i=i+1)expected_rs[i]=(i>=7 && i!=23);
25     expected_rs[38]=1;expected_rs[39]=1;
26 end
27 always @(negedge lcd_e) if(!rst_p && count<40)begin
28     if(lcd_rw!=0||lcd_rs!=expected_rs[count]||lcd_data!=expected_data[count])
29         $fatal(1,"index=%0d rs=%b data=%h",count,lcd_rs,lcd_data);
30     count=count+1;
31     if(count==40)begin $display("LAB3_LCD_PASS bytes=%0d",count);$finish;end
32 end
33 initial begin
34     $dumpfile("wave.vcd");$dumpvars(0,tb_character_lcd);
35     repeat(3)@(posedge clk_50mhz);rst_p=0;
36 end
```

tb_character_lcd.sv · 37–38행

```
37     initial begin #50000;$fatal(1,"timeout count=%0d",count);end  
38 endmodule
```

lab3_character_lcd.xdc · 1-17행

```
1 # Combo II-DLD S75 / MAIN CLOCK F = 50 MHz.
2 set_property PACKAGE_PIN B6 [get_ports clk_50mhz]
3 set_property PACKAGE_PIN K4 [get_ports rst_p]
4 set_property PACKAGE_PIN A6 [get_ports lcd_e]
5 set_property PACKAGE_PIN G6 [get_ports lcd_rs]
6 set_property PACKAGE_PIN D6 [get_ports lcd_rw]
7 set_property PACKAGE_PIN A4 [get_ports {lcd_data[0]}]
8 set_property PACKAGE_PIN B2 [get_ports {lcd_data[1]}]
9 set_property PACKAGE_PIN C3 [get_ports {lcd_data[2]}]
10 set_property PACKAGE_PIN D4 [get_ports {lcd_data[3]}]
11 set_property PACKAGE_PIN A2 [get_ports {lcd_data[4]}]
12 set_property PACKAGE_PIN C5 [get_ports {lcd_data[5]}]
13 set_property PACKAGE_PIN C1 [get_ports {lcd_data[6]}]
14 set_property PACKAGE_PIN D1 [get_ports {lcd_data[7]}]
15 set_property IOSTANDARD LVCMOS33 [get_ports *]
16 create_clock -name clk_50mhz -period 20.000 [get_ports clk_50mhz]
17 set_false_path -from [get_ports rst_p]
```

VS Code에서 Icarus 실행

1. File → Save All.
2. Terminal → Run Task... → 01 Check tools.
3. Git·Python·iverilog·vvp 경로와 버전을 확인한다.
4. Terminal → Run Task... → 02 Simulate.
5. LAB3_LCD_PASS bytes=40와 종료 시각을 기록한다.
6. 03 Open waveform으로 build/sim/wave.vcd를 연다.

VS Code · 프로젝트와 파일

실제 화면 캡처 자리

ID: 06_character_lcd-vscode-workspace

FPGA.code-workspace를 연 뒤 Explorer에 src.sim.constraints.tools와 작성 파일이 보이게 캡처한다.

VS Code · 시뮬레이션 PASS

실제 화면 캡처 자리

ID: 06_character_lcd-vscode-pass

02 Simulate 실행 뒤 LAB3_LCD_PASS bytes=40와 종료 시각이 함께 보이게 캡처한다.

VS Code · 파형

실제 화면 캡처 자리

ID: 06_character_lcd-vscode-wave

TB top을 펼쳐 clk_50mhz·rst_p·주요 입력·내부 상태·출력을
추가하고 경계 조건이 보이게 확대한다.

파형에서 설명할 내용

Enable 하강 에지에서 초기화 명령과 두 줄 데이터, 총 40바이트의 순서와 RS를 검사한다.

입력을 바꾼 시점, 유효 상승 에지, 상태 갱신 직후를 구분한다.

파라미터를 줄인 TB 시간과 실제 50 MHz 보드 시간을 혼동하지 않는다.

수정 실험

둘째 줄 문자열 한 글자를 바꾸어 ASCII 바이트와 표시 결과를 비교한 뒤 복구한다.

정상 PASS와 파형을 먼저 보관한 뒤 한 항목만 수정한다. 예상값, 실제 FAIL 또는 변화, 복구 PASS를 3행 표로 기록한다.

TB의 기대값을 RTL 오류에 맞춰 바꾸지 않는다.

Vivado 2026.1 프로젝트 생성

1. Create Project → RTL Project → Do not specify sources at this time.
2. Part에 xc7s75fgga484-1을 정확히 선택한다.
3. Add Sources에 src의 RTL 전부, Add Simulation Sources에 TB를 추가한다.
4. Add or Create Constraints에 XDC를 추가한다.
5. Design Sources의 top을 lab3_character_lcd으로 설정한다.

Vivado · 소스와 top

실제 화면 캡처 자리

ID: 06_character_lcd-vivado-sources

Sources 창에서 RTL, Simulation Sources의 TB, Constraints의 XDC
와 top module이 함께 보이게 캡처한다.

Vivado XSim

Flow Navigator → Simulation → Run Behavioral Simulation.

Tcl Console의 LAB3_LCD_PASS bytes=40를 확인한다. Objects/Wave에서 입력과 출력을 추가하고 Zoom Fit으로 전체 실행을 본다.

Icarus와 XSim은 같은 RTL·TB를 실행해야 한다. 서로 다른 사본을 등록하지 않는다.

Vivado · XSim PASS와 파형

실제 화면 캡처 자리

ID: 06_character_lcd-xsim-pass

Tcl Console PASS, waveform 이름, 시간축, 핵심 입출력이 한 화면에 보이게 캡처한다.

핀과 클록 제약 확인

Open Elaborated Design 또는 I/O Planning에서 top port와 PACKAGE_PIN을 대조한다.

Reports → Timing → Report Clocks에서 clk_50mhz 주기 20.000 ns를 확인한다.

XDC를 저장한 뒤 Run Synthesis → Run Implementation → Generate Bitstream 순서로 실행한다.

Vivado · 구현과 bit 생성

실제 화면 캡처 자리

ID: 06_character_lcd-vivado-bit

Implementation 완료, Timing Summary의 WNS, DRC 오류 0건,
bitstream 생성 완료가 확인되는 화면을 캡처한다.

보드에서 확인

전원을 끈 상태에서 외부 부품 결선과 공통 GND를 먼저 확인한다.

Hardware Manager → Open Target → Auto Connect → Program Device에서
생성한 bit를 기록한다.

5 V/3.3 V 결선과 대비를 먼저 확인하고 FPGA LAB3 / LCD CONTROLLER
문자열을 확인한다.

사진에는 보드 전체와 사용 핀이, 영상에는 입력 조작과 출력 변화가 함께
보이게 한다.

실험 전 레포트

1. 동작 원리와 핵심 파라미터 계산을 자기 말로 설명한다.
2. RTL·TB·XDC 각 파일의 역할과 top module을 적는다.
3. TB 자극과 기대 결과를 표로 정리한다.
4. Icarus PASS·파형과 수정/복구 결과를 넣고 경계 조건을 설명한다.
5. 보드에서 확인해야 할 항목과 예상 결과를 적는다.

실험 후 레포트와 제출

1. XSim PASS·파형, DRC·timing·bitstream 결과를 기록한다.
2. 실제 보드 관찰, 계산값과 실측의 차이, 원인을 설명한다.
3. 결선이 보이는 사진과 동작 영상 링크를 첨부한다.
4. 자신의 GitHub 저장소 URL과 최종 commit hash를 적는다.
5. src·sim·constraints·simulation.json만 최종 원본으로 관리하고 생성물은 제외한다.

완료 체크

- ☐ v2.0.2 태그에서 새 프로젝트를 만들었다.
- ☐ RTL·TB·XDC를 직접 작성하고 설명할 수 있다.
- ☐ Icarus와 XSim에서 같은 PASS를 확인했다.
- ☐ 핀·20 ns 클록·DRC·timing을 확인하고 bit를 만들었다.
- ☐ 보드 동작 사진·영상과 GitHub 링크를 정리했다.
- ☐ 실험 전·후 레포트에 결과와 한계를 구분해 기록했다.